

exploits the characteristics of vector query processing to efficiently pipeline data transmission from host memory to GPU memory and query processing in GPU.

Three primary technical challenges must be overcome to realize RUMMY, each pertaining to a different aspect: transmission, computation, and pipelining. First, redundant transmission occurs when processing vector queries in a batch. Due to limited GPU memory, the same data subset is transmitted and evicted repeatedly for different queries in the same batch. Leveraging the fact that each subset’s processing is independent, we design *cluster-based retrofitting* to restructure the query plan for each vector query and *completely* eliminate redundant transmission.

Second, the Streaming Multiprocessors (SMs) in the GPU suffer from low utilization due to pipelining-caused load imbalance. Specifically, pipelining segregates the query processing into groups, each corresponding to a GPU kernel. A kernel comprises a grid of thread blocks, and each block is executed within an SM. A kernel for a small group is launched with a small grid of thread blocks, resulting in idle SMs (i.e., spatial underutilization). Besides, the discrepancy between the size of different data parts causes stragglers—some thread blocks execute longer than others (i.e., temporal underutilization). To maximize utilization, we introduce *dynamic kernel padding* with *cluster balancing*. Cluster balancing equalizes data part sizes offline. Dynamic kernel padding fills idle SMs with padding thread blocks online to enhance GPU occupancy.

Third and most importantly, directly applying pipelining suffers from limited overlapping between transmission and computation, and has high pipelining overhead due to frequent API invocations and synchronizations. The problem is exacerbated by that the pipelining plan can only be decided at runtime when queries arrive, as the quality of a particular plan is query-dependent. We design *query-aware reordering and grouping*, which dynamically reorders the query plan and divides the plan into groups with two algorithms. The reordering algorithm hides small transmission operations with large computation operations as many as possible to maximize the overlapping. The grouping algorithm finds the best trade-off between pipelining efficiency and pipelining overhead. The two algorithms are lightweight with negligible runtime overhead. We prove that the two algorithms achieve optimal pipelining performance, respectively.

In addition, we also tailor the GPU memory management for vector query processing. RUMMY pre-allocates GPU memory and re-allocates the memory to each transmission task, without involving the general-purpose but costly GPU memory manager. We also exploit the vector query processing to reduce GPU memory fragmentation and cache misses.

We remark that the runtime *reordered pipelining* technique in RUMMY is distinct from canonical pipelining techniques adopted in microprocessor architectures [27, 28], DL systems [29, 30], and task schedulers [31, 32]. Those techniques rely on dependent and deterministic execution flows. For

instance, DL models have a pre-defined layer-by-layer computation graph, which enables solutions like PipeDream [30] and PipeSwitch [29] to compute a pipelining plan *offline*. Besides, to ensure correctness, the deterministic computation graph cannot be *reordered*. In contrast, a vector query plan is *non-deterministic* and can be executed in any order since the processing of each subset is *independent*. The key novelty of RUMMY is that it leverages the independent and non-deterministic nature of vector query processing to dynamically decide the pipelining plan with *reordering at runtime*, and achieves optimal parallelism between transmission and computation.

In summary, we make the following contributions.

- We present RUMMY, to the best of our knowledge, the first GPU-accelerated vector query processing system that achieves high performance and supports large vector datasets beyond GPU memory.
- We introduce a new runtime *reordered pipelining* technique, which leverages *cluster-based retrofitting*, *dynamic kernel padding* with *cluster balancing*, and *query-aware reordering and grouping* to efficiently pipeline data transmission and query processing.
- We implement a RUMMY prototype. The evaluation shows that RUMMY outperforms IVF-GPU with CUDA unified memory by up to $135\times$. Compared to the state-of-the-art CPU solution (with 64 vCPUs), RUMMY (with one NVIDIA A100 GPU) achieves up to $23.1\times$ better performance and is up to $37.7\times$ more cost-effective.

2 Background and Motivation

In this section, we first introduce the background of vector query processing. We then describe GPU-based solutions to accelerate vector query processing. Finally, we summarize the challenges to support billion-scale datasets for GPU-based vector queries, which motivate the design of RUMMY.

2.1 Vector Query Processing

DL models convert unstructured data into high-dimensional feature vectors to serve applications [4–12, 21, 22]. These vector datasets [24, 25] are utilized to construct a vector database [16–20] as the persistence and long-term memory. Specifically, RAG leverages these datasets to enhance the quality of LLM-based AI applications. For instance, TEXT1B [24] contains one billion data entries from both textual and visual modalities, which can be used to enhance multimodal LLMs. Vector queries are referred to as the memory retrieval part in RAG and are widely used in emerging LLM-based AI applications [33–35].

Specifically, a vector query is to find the top- k nearest neighbors (KNN) in a vector dataset that are most similar to the given vector. KNN returns the exact top- k results and requires searching on the entire dataset. KNN becomes impractical for large datasets due to high query latency. Approximate top- k nearest neighbor (ANN) search trades query accuracy

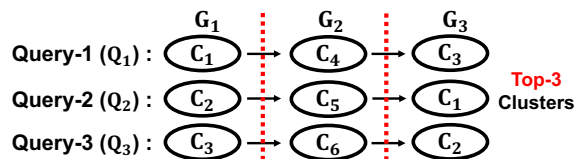


Figure 1: Query plan for a batch of three queries.

for query latency. Previous works [36] show that ANN can achieve 99% accuracy while only searching on 1% of the dataset. Even for accuracy-sensitive scenarios, ANN can meet the requirements of applications and outperform KNN by hundreds of times. Since DL models inherently introduce errors when generating vectors, ANN is widely used in existing vector databases for large datasets [16, 18, 19, 37–40].

The basic idea of ANN search is to build an index structure [41–44] to sample a subset of the dataset for answering a vector query. There are two representative ANN indexes, inverted file index (IVF) [41, 45, 46] and graph index [42, 47, 48]. While graph index is more accurate, it is more expensive to build and maintain due to its large memory footprint and building time. When confronting billion-scale datasets, IVF is proved to be more efficient and accurate than the state-of-the-art graph index [48] with the same memory consumption according to recent work [46]. Thus, IVF is more suitable for very large datasets. While IVF has many variants, their general workflow is similar. At offline, IVF trains a list of clusters ($\{C_1, C_2, \dots, C_l\}$) by k-means clustering [49], and each vector is assigned to the closest cluster. Online, it performs pairwise comparison between the query vector and each vector in the top- n closest clusters to produce the approximate top- k nearest neighbor vectors. The number of sampled clusters n is a configurable parameter to make a tradeoff between accuracy and latency.

For example, Figure 1 shows a query plan of the batch of three queries ($\{Q_1, Q_2, Q_3\}$). IVF selects top-3 nearest clusters (i.e., $n = 3$) for each query, e.g., $\{C_1, C_4, C_3\}$ for Q_1 . Q_1 is compared with each vector in the three clusters, in which the top- k nearest vectors are returned as the result.

2.2 GPU Acceleration

Vector query processing is compute-intensive and involves massive vector operations. GPUs are a natural choice to accelerate vector processing [23]. The pairwise distance computation between high-dimensional vectors is a good fit for the GPU architecture. A detailed comparison between CPUs and GPUs on IVF and graph index in terms of performance and cost is listed in Appendix A.2, which demonstrates the benefits of GPU-based vector query processing systems.

Vector query processing in GPUs. A GPU consists of many SMs, each including numerous CUDA cores for extensive vector operations. Such hardware architecture involves a hierarchical parallel computation model [50]: each computation consists of multiple CUDA kernels; each CUDA kernel in-

cludes a grid of thread blocks; and each block comprises a number of threads. A block of threads can be executed simultaneously within one SM, where each thread is executed on a CUDA core. To process a vector query in a GPU, the computation of the query on one cluster (e.g., $Q_1 \rightarrow C_1$ in Figure 1) is a thread block. The computations of different queries on their corresponding clusters (e.g., $\{Q_1 \rightarrow C_1, Q_2 \rightarrow C_2, Q_3 \rightarrow C_3\}$) form a grid of thread blocks, i.e., a CUDA kernel.

Limitation: GPU memory capacity. Existing GPU-based vector query systems [23] load the entire dataset in the GPU. Today’s large vector datasets [24, 25] contain billions of vectors, which have a memory footprint of hundreds of GB. GPU memory is too limited compared to large datasets. For example, NVIDIA A100 GPUs are equipped with up to 80GB of memory, while the size of the TEXT1B dataset [24] is 750GB. It requires at least ten NVIDIA A100 GPUs to store the TEXT1B dataset. Using multiple GPUs causes a mismatch between GPU compute resources and GPU memory resources. In particular, as we have explained in §2.1 and Figure 1, ANN search only performs vector operations on a small subset of the dataset, which exacerbates this mismatch and causes low utilization of expensive GPU resources.

The natural idea is to expand GPU memory with host memory. For example, the dataset is stored in host memory and is divided into multiple parts, each of which can fit GPU memory. To process a query, each part is transmitted from host memory to GPU memory *in rotation*. After iterating over all parts, the intermediate results of each part are aggregated to produce the final result. This solution has a long transmission time, and because it performs transmission and computation sequentially, it has low GPU utilization.

Another strawman solution is to integrate existing GPU-based query systems with CUDA unified memory [26]. CUDA unified memory automatically handles data transmission at runtime, and performs parallel transmission and computation to fully utilize the GPU copy engine and kernel engine. However, it is a general memory swapping technique that is unaware of vector queries. It incurs massive GPU memory page faults for data transmission. Besides, the overlapping between transmission and computation is also limited as it is agnostic of vector queries. Therefore, applying CUDA unified memory introduces a large gap from the ideal performance.

2.3 Challenges

Opportunity: pipelining transmission and computation. We can pipeline transmission and computation to increase GPU utilization and improve performance. Specifically, we divide the query processing into groups, e.g., $\{G_1, G_2, G_3\}$ in Figure 1. The transmission of one group can be pipelined with the computation of the preceding group. Figure 2(a) shows how to pipeline the three groups in Figure 1, where PCIe represents data transmission from host to GPU memory over PCIe, and GPU represents computation in GPU. The transmission of G_2 is pipelined with the computation of the

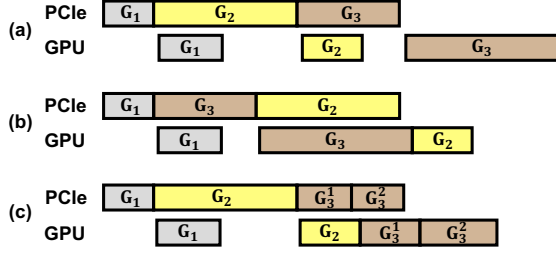


Figure 2: Examples for the pipelining plans (ignore data reuse).

preceding group G_1 , partly hiding the transmission overhead. We summarize three challenges to exploit this opportunity and achieve optimal pipelining efficiency, which focus on three system aspects: transmission, computation, and pipelining.

Challenge 1: cross-query redundant transmission. The first challenge is that processing a batch of vector queries introduces redundant transmission. As Figure 1 shows, if the GPU memory capacity equals three clusters and the execution order is $[G_1, G_2, G_3]$. Initially, the GPU is empty. The system transmits $G_1 = \{C_1, C_2, C_3\}$ to the GPU, which fills up the GPU memory. Next, to transmit $\{C_4, C_5, C_6\}$ for G_2 , it first evicts $\{C_1, C_2, C_3\}$ and then starts the transmission. As for G_3 , it clears the GPU memory again and then transmits $\{C_1, C_2, C_3\}$ back to GPU memory. In total, the order $[G_1, G_2, G_3]$ transmits nine clusters, among which C_1 , C_2 and C_3 are transmitted twice each. Alternatively, swapping the order between G_2 and G_3 only needs to transmit six clusters. In this case, the system reuses the data $\{C_1, C_2, C_3\}$ for G_3 after processing G_1 . Reordering G_2 and G_3 eliminates the redundant transmission.

Challenge 2: spatial and temporal GPU underutilization. The second challenge is the SMs in the GPU have low utilization due to the load imbalance of thread blocks. The computation of G_1 in Figure 1 corresponds to a kernel that contains three thread blocks (i.e., $\{Q_1 \rightarrow C_1, Q_2 \rightarrow C_2, Q_3 \rightarrow C_3\}$). $Q_i \rightarrow C_j$ represents the thread block of the computation of query Q_i on cluster C_j . Each thread block is executed within one SM and cannot be migrated to other SMs [50]. As shown in Figure 3, when processing G_1 , there are only three thread blocks to run, if the GPU has four SMs, then one SM (i.e., SM_4) is idle, causing *spatial underutilization*. Besides, because the clusters have different sizes, the thread blocks for larger clusters take a longer time to run, i.e., stragglers. As shown in Figure 3, SM_2 and SM_3 are idle after completing their thread blocks, leading to *temporal underutilization* as they wait for SM_1 to finish. In summary, spatial underutilization stems from a mismatch between the number of SMs and thread blocks, while temporal underutilization is caused by cluster heterogeneity.

Challenge 3: transmission and computation overlapping. The third challenge is to maximize the overlapping between transmission and computation in the pipeline. The overlapping is affected by both the ordering and granularity of the

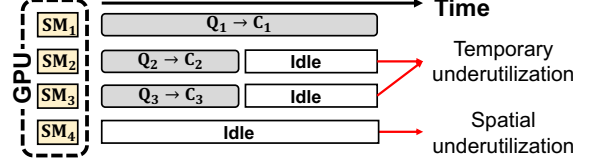


Figure 3: Spatial and temporal GPU underutilization.

groups in the pipeline. We illustrate each of them in Figure 2, and we do not consider data reuse in this example. Figure 2(a) shows a primitive pipelining plan based on the execution order $[G_1, G_2, G_3]$. First, as shown in Figure 2(b), if we reorder G_2 and G_3 , then the transmission of G_2 is completely hidden by the computation of G_3 , which increases the overlapping. Second, as shown in Figure 2(c), dividing G_3 into two smaller groups, G_3^1 and G_3^2 , also improves the overlapping, because finer-grained grouping can hide more transmission operations with computation operations. However, finer-grained grouping also introduces more system overhead due to frequent API invocations and synchronizations.

The challenge is exacerbated by the runtime nature of the problem. The pipelining plan depends on the input vector queries, as different queries process different clusters, which affects the ordering and grouping decisions of the pipeline. Let N be the number of thread blocks. The search space of exhaustive search contains $O(N! \times 2^N)$ choices, as there are $O(N!)$ ordering cases and $O(2^N)$ grouping cases. It is challenging to find the best plan among these choices. In addition, the system also needs an accurate *profiler* to precisely estimate the transmission and computation time for a given group at runtime.

3 RUMMY Overview

We present RUMMY, a GPU-accelerated vector query processing system to support large vector datasets beyond GPU memory. RUMMY exploits the characteristics of vector queries to achieve fast and cost-effective query processing. RUMMY achieves so by pipelining transmission and computation through a novel *reordered pipelining* technique. This technique eliminates redundant data transmission (§4.1), maximizes GPU utilization (§4.2), and finds out optimal pipelining plans with negligible runtime overhead (§4.3). Here we provide a brief overview of RUMMY as Figure 4 shows. RUMMY consists of an offline part and an online part.

Offline. RUMMY first builds a primitive IVF index for the dataset, which can be extended into many variants of IVF. IVF divides the vectors into a few clusters. RUMMY extends the index building with *cluster balancing* to alleviate the straggler problem (§4.2). The dataset is stored in host memory and is transmitted to the GPU memory over PCIe online under the control of the GPU memory management system (§4.4). RUMMY also builds a *profiler* offline to measure the computation and transmission time (§4.3.1) for online decision making.